



Artificial Intelligence:

Mimicking the intelligence or behavioural pattern of humans or any other living entity.

Machine Learning:

A technique by which a computer can "learn" from data, without using a complex set of different rules. This approach is mainly based on training a model from datasets.

Deep Learning:

A technique to perform machine learning inspired by our brain's own network of neurons.

Seidenberg School Of Computer Sciences | Pace University

AI – ML – DL

AI Initiatives – AI 'Winters' – AI New Approaches

The Perceptron Model – MLP (Multi-layer ...)

Prof. Tassos H. Sarbanes

Course's Goal and Format

Goals

- Practical, hands-on intro to DL/ML
- Application heavy; Theory lite but know enough...
- Assumes great knowledge of Python/R

Format

- Have a laptop (w/ all required packages installed)
- Instead, we'll do lecture & live coding sessions

Objective and Approach

Goal is to give you the concepts, the intuitions, and the tools you need to actually implement programs capable of learning from data.

Cover a large number of techniques, from the simplest and most commonly used (linear regression) to some of the Deep Learning techniques that regularly win competitions.

Rather than implementing our own toy versions of each algorithm, we will be using actual production-ready Python frameworks (the power of Open Source initiative!)

Scikit-Learn

Scikit-Learn is very easy to use, yet it implements many Machine Learning algorithms efficiently, so it makes for a great entry point to learn Machine Learning.

Scikit-Learn vs TensorFlow

TensorFlow is a more complex library for distributed numerical **computation** using **data flow graphs**.

It makes it possible to train and run very large neural networks efficiently by distributing the computations across potentially thousands of multi-GPU servers.

TensorFlow was created at Google and supports many of their large-scale Machine Learning applications.

It was open-sourced in November 2015.

**The hard problems are easy,
and
the easy problems are hard.**

<Q> Is math easy?

<Q> Is gardening hard?

Moravec's Paradox and AI

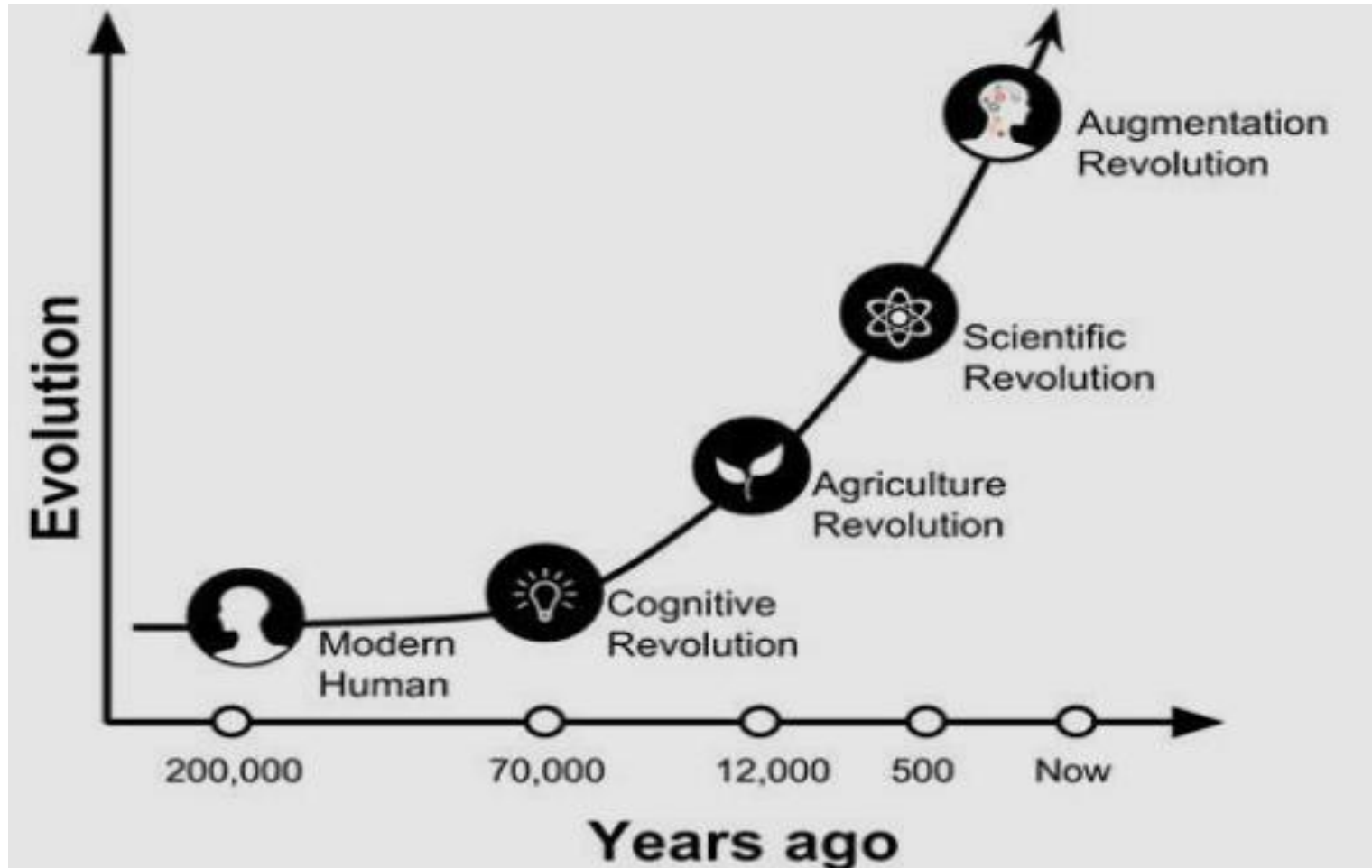
Moravec's paradox is the observation by artificial intelligence and robotics researchers that, contrary to traditional assumptions, reasoning requires very little computation, but sensorimotor skills require enormous computational resources. - Wikipedia

Moravec wrote, “It is comparatively easy to make computers exhibit adult level performance on intelligence tests or playing checkers, and difficult or impossible to give them the skills of a one-year-old when it comes to perception and mobility.”

Hans Peter Moravec of Robotics Institute of Carnegie Mellon University

Is Math Easy and Gardening Hard?

Why math is easy for AI but gardening is not: Moravec's paradox

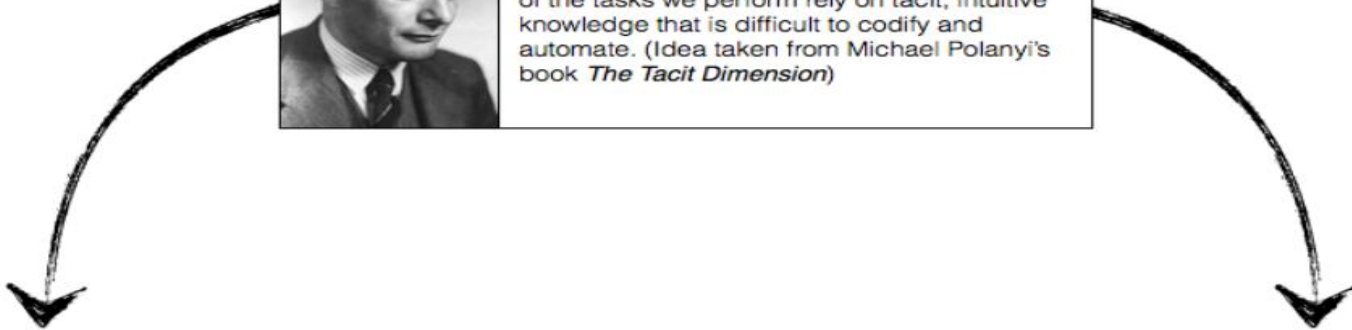


Polanyi's Paradox – The Tacit Dimension



Polanyi's Paradox

We can know more than we can tell, i.e. many of the tasks we perform rely on tacit, intuitive knowledge that is difficult to codify and automate. (Idea taken from Michael Polanyi's book *The Tacit Dimension*)



Environmental Control

You overcome Polanyi's Paradox by manipulating the environment in such a way that it is easier for machines to perform the task, e.g. Kiva robots and warehouse design



Machine Learning

You try to get the machine to mimic expert human judgment (which often relies on tacit knowledge and heuristics). You do this by getting the computer to infer its own rules from a series of trials on a large dataset, e.g. IBM's Watson



Tacit vs Explicit

Polanyi's Paradox: (We know more than we can tell!)

Will humans maintain any advantage over machines?

Michael Polanyi: British-Hungarian philosopher

What is Artificial Intelligence (AI)?

Inventors have long dreamed of **creating machines that think**. This desire dates back to at least the time of ancient Greece.

Today, artificial intelligence(AI) is a thriving field with many practical applications and active research topics. We look to intelligent software to automate routine labor, understand speech or images, make diagnoses in medicine and support basic scientific research

Chapter 1 / Introduction

<https://www.deeplearningbook.org/contents/intro.html>

Early Days of AI

Early days of AI, tackled, solved problems that are intellectually difficult for humans but relatively straight-forward for computers — problems described by a list of formal, mathematical rules.

True **challenge to artificial intelligence** proved to be **solving the tasks that are easy for people to perform but hard for people to describe formally**—problems people solve intuitively, that feel automatic, e.g. recognizing spoken words or faces in images.

Early days about a solution to these more intuitive problems. Solution is to allow computers to learn from experience and understand the world in terms of a hierarchy of concepts, with each concept defined through its relation to simpler concepts

Early Days of AI – Approach

By **gathering knowledge from experience**, this approach avoids the need for human operators to formally specify all the knowledge that the computer needs.

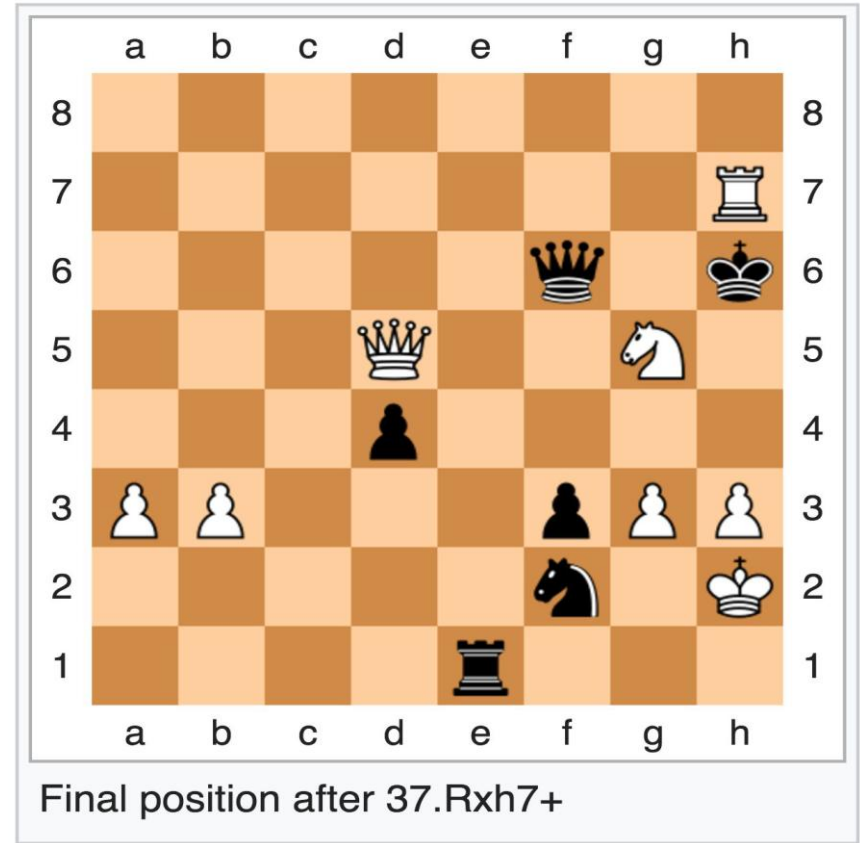
The **hierarchy of concepts** enables the computer to learn complicated concepts by building them out of simpler ones.

If we draw a **graph** showing how these concepts are built on top of each other, the **graph is deep**, with **many layers**. For this reason, we call this approach to AI **Deep Learning**

Early AI Successes!

Many of the early successes of AI took place in relatively sterile and formal environments and did not require computers to have much knowledge about the world. For example, **IBM's Deep Blue** chess-playing system defeated world champion Garry Kasparov in 1997

Deep Blue vs. Garry Kasparov - 1997



Chess Playing -- History

For much of modern history, chess playing has been seen as a "litmus test" of the **ability for computers to act intelligently**. In 1770, the Hungarian inventor Wolfgang von Kempelen unveiled "The Turk", a (fake) chess-playing machine

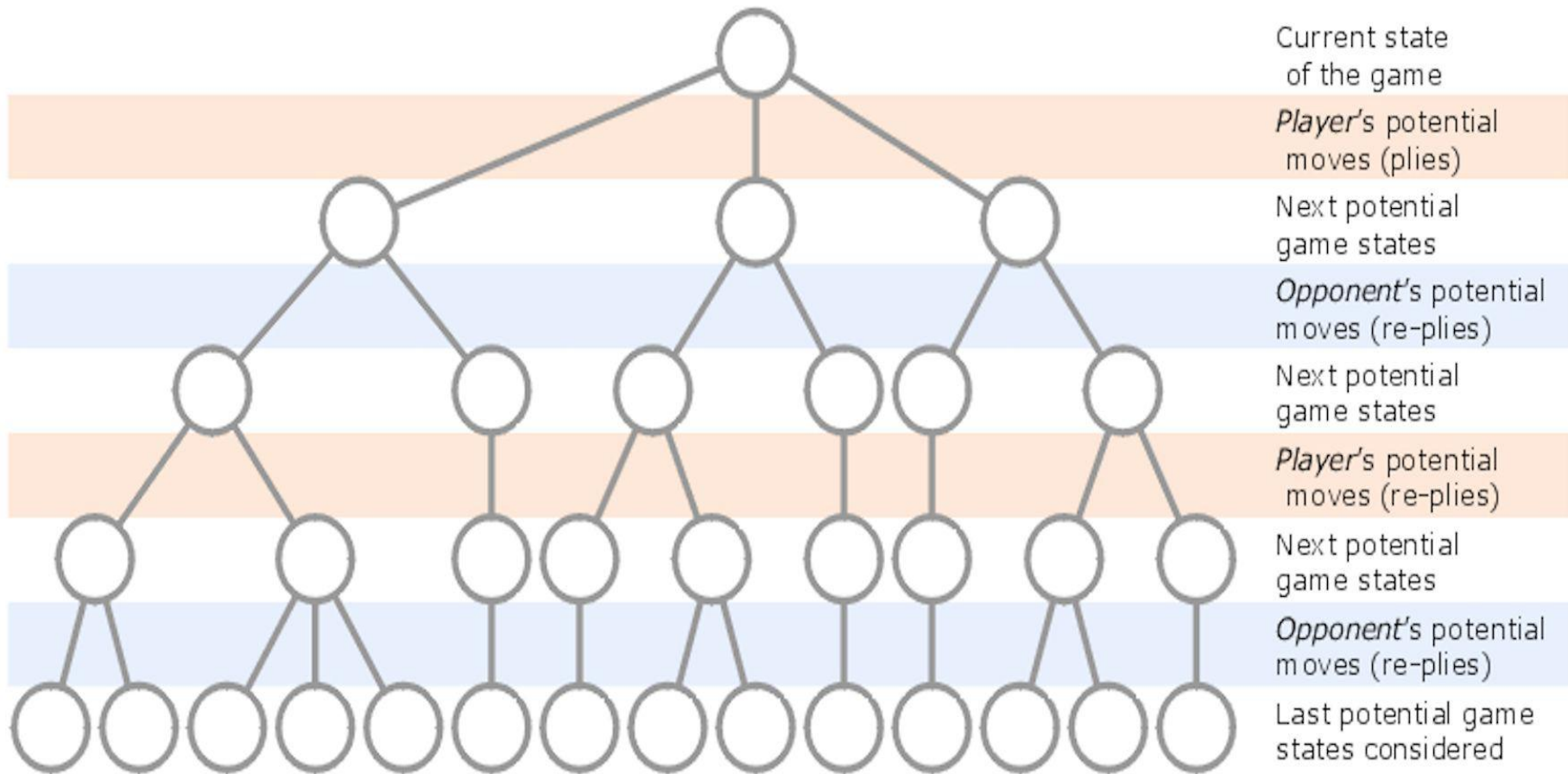
With the advent of computers (1940s), researchers began the first serious attempts at making an intelligent chess-playing machine. In 1950, **Claude Shannon** published a groundbreaking paper entitled "Programming a Computer for Playing Chess", which first put forth the idea of a **function** for **evaluating** the efficacy of a particular move and a "**minimax**" algorithm which took advantage of this evaluation function by **taking into account the efficacy of future moves** that would be made available by any particular move. This work provided a framework for all future research in computer chess playing.

<Q> Do you remember **Claude Shannon** and 3 more during the 'hot summer of 1955 at Dartmouth?

"The 7 Aspects of Artificial Intelligence!"

Minimax, Claude Shannon - 1950(!)

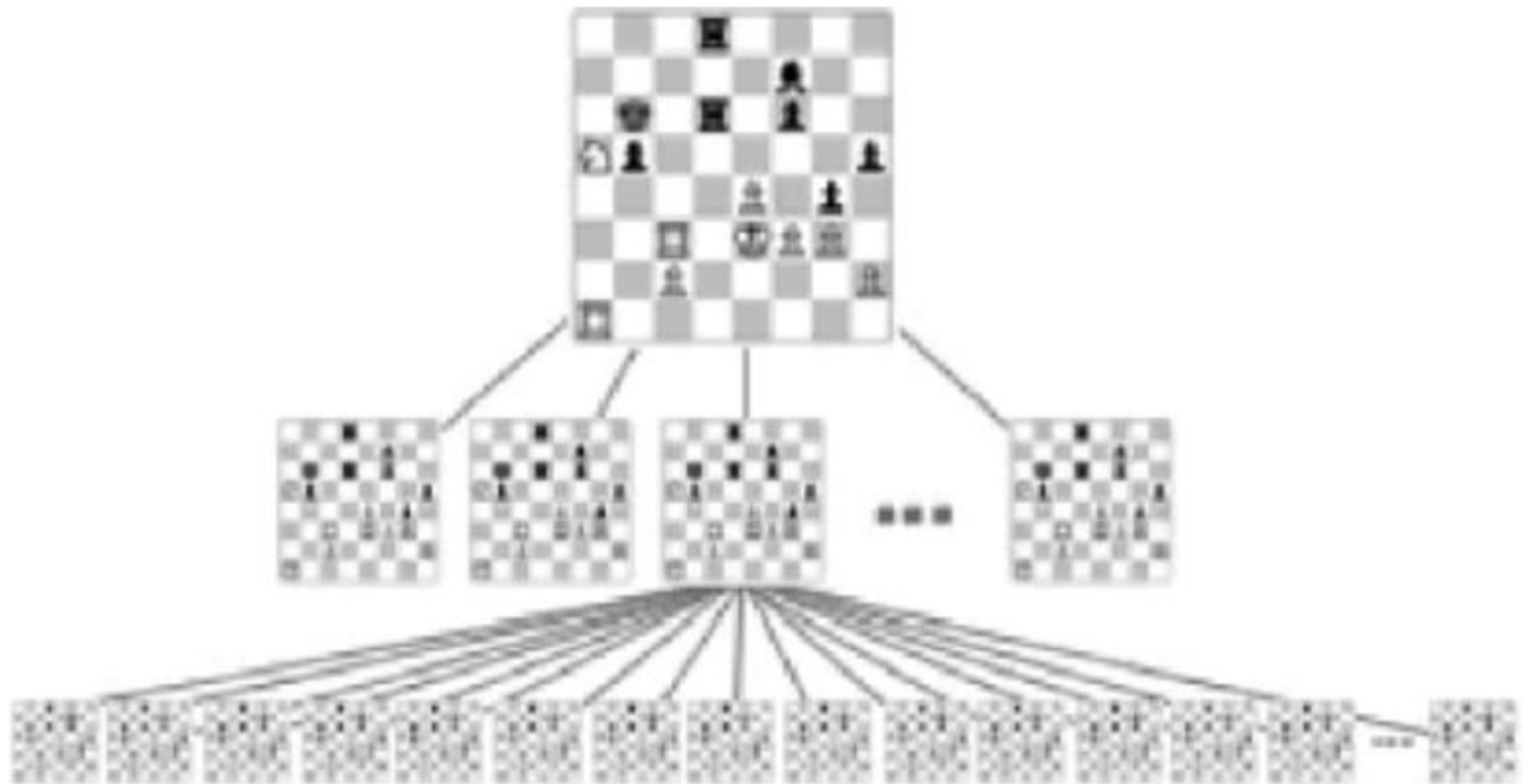
<https://vision.unipv.it/IA1/ProgrammingaComputerforPlayingChess.pdf>



Shannon: The Father of Information Theory
Entropy (Information Gain)

Early AI – AI Techniques / Tree Search

Basic model of chess -> **Tree Search problem (Data Representation)**, where each state is a particular arrangement of the pieces on the board and the available actions correspond to the legal chess moves for the current player in that arrangement. E.g. "slice" of such a tree is in this figure:

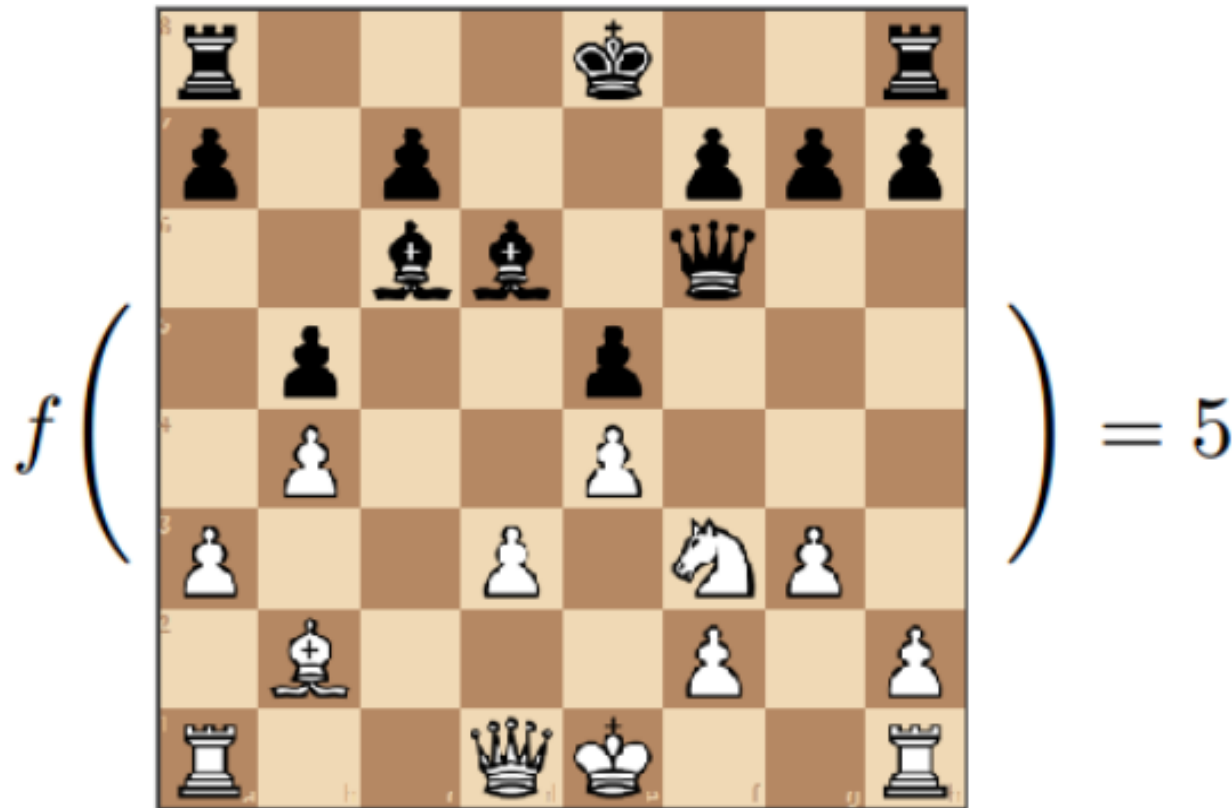


Early AI – AI Techniques / The Evaluation Function

As put forth in Shannon's paper, the primary ingredient in a chess-playing program is the **evaluation function**. Since we can't look forward all the way to the end of the game and see if a particular move will win (especially since we don't know what the other player will do during their turns!), we must **create a function which takes in a state of the game** (in our case, a board arrangement) and boils it down to a real-number evaluation of the state.

E.g., function could give higher scores to board states in which the player of interest has more of their pieces on the board than the opponent. Want the **function to assign** an extremely high **score** (perhaps even infinity) to the board arrangement in which the opponent's king is in 'checkmate', meaning that the player of interest is guaranteed to win the game.

Early AI – AI Techniques / The Evaluation Function (Example)



Chessboard: Certain state of the game ...

Early AI – AI Techniques / The Minimax Algorithm

Given an evaluation, all left is a way of choosing which move to take. Looking ahead one step and choosing a move which leads to board arrangement w/ highest evaluation score is good baseline, be smarter take into account the actions opponent could take once we've moved.

This intuition leads to the "**Minimax algorithm**", so-called because we choose the action which *minimizes our maximum possible "loss"* from making a particular move.

Specifically, for each move we make we look ahead as many steps as our *computing power will allow (*)* and examine all the possible moves our opponent could make in each of their future turns, given that we've made our original move. We then take the maximum "loss" (equivalently, the minimum of our evaluation function) that our opponent could induce for us via their moves, choose a move that minimizes this maximum

Early AI – AI Techniques / Heuristics & Optimizations

Having an evaluation function and an implementation of the minimax algorithm, one can design an effective chess-playing program.

Scientists went a step further, implementing "**heuristics**", simple rules which can **cut down on computation time**, along with **optimizations of the minimax algorithm**, given the specific structure of chess.

E.g., heuristic: if a move leads to the player's king being in checkmate, then algorithm should not look any farther down that path of the game tree, since we know the player will never want to make that move.

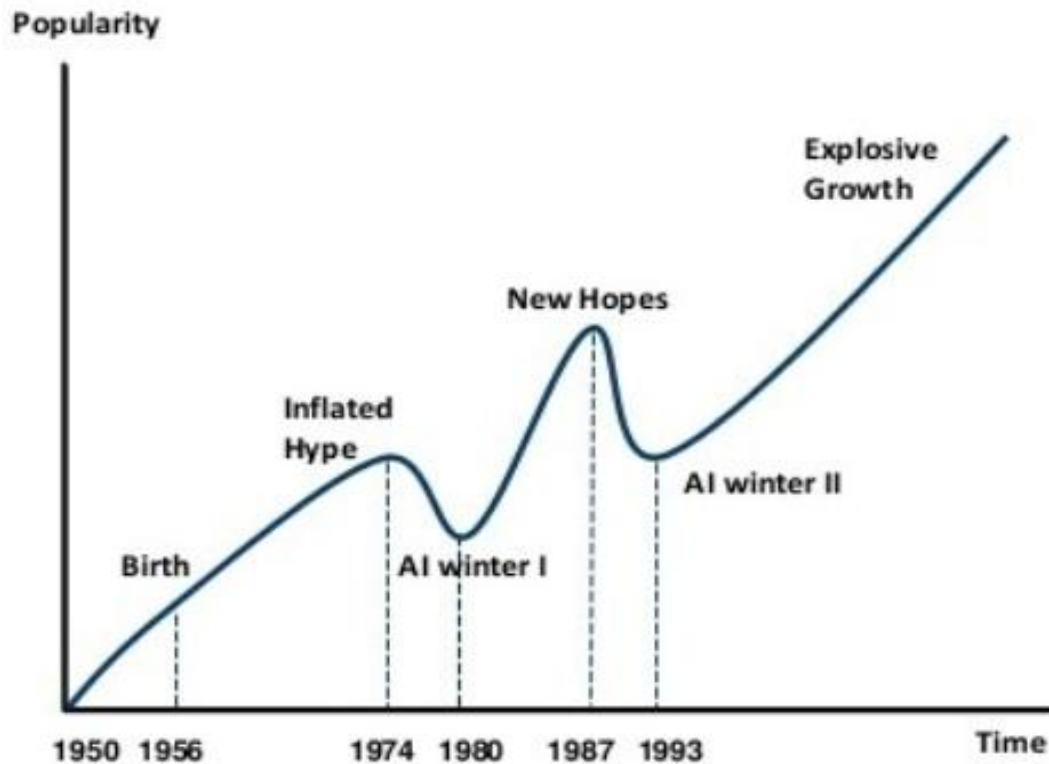
A popular optimization of minimax is known as Alpha-beta pruning, wherein any move for which another move has already been discovered that is guaranteed to do better than it is eliminated.



<https://en.wikipedia.org/wiki/Alpha-beta-pruning>

AI 'Winters' -- History

The concept of AI was first proposed almost 70 years ago (hot summer of 1955 @ Dartmouth). What happened in the AI research community over this time period, why did it take so long for the breakthroughs to occur?

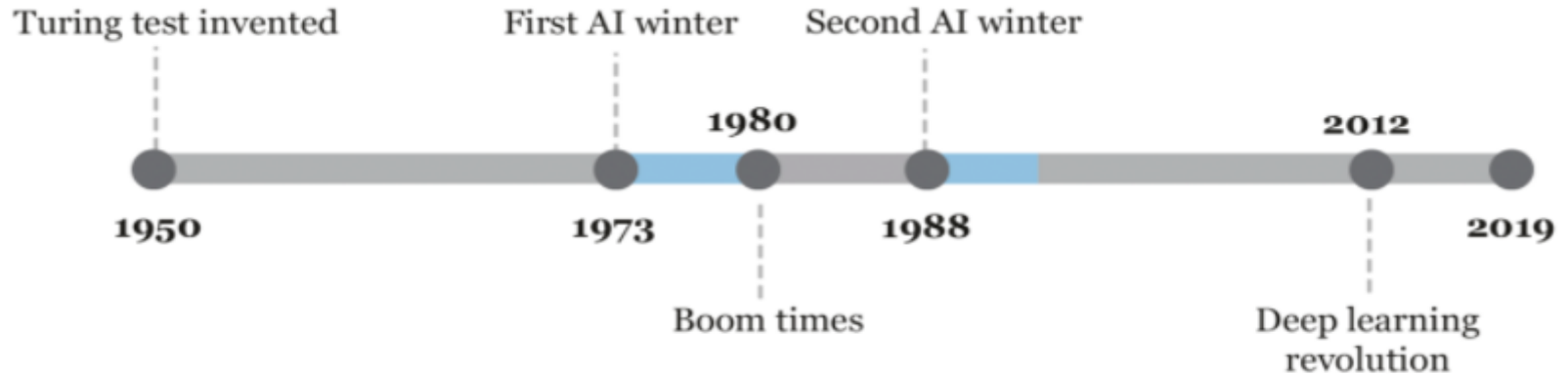


Timeline of AI Development

- **1950s-1960s:** First AI boom - the age of reasoning, prototype AI developed
- **1970s:** AI winter I
- **1980s-1990s:** Second AI boom: the age of Knowledge representation (appearance of expert systems capable of reproducing human decision-making)
- **1990s:** AI winter II
- **1997:** Deep Blue beats Gary Kasparov
- **2006:** University of Toronto develops Deep Learning
- **2011:** IBM's Watson won Jeopardy
- **2016:** Go software based on Deep Learning beats world's champions

AI has a long history of being "The Next Big Thing"...

AI Winters Freeze Progress



1974-1980

First AI winter

Limited applicability of AI leads to funding pullback in the U.S. and abroad.

1969: Researchers Marvin Minsky and Seymour Papert published *Perceptrons*, an influential book pointing out the ways early neural networks failed to live up to expectations.

1970-1974: DARPA cut its funding as enthusiasm wore thin.

1974: The Lighthill report, compiled by researcher James Lighthill for the British Science Research Council, stated: "In no part of the field [of AI] have the discoveries made so far produced the major impact that was then promised."

1987-1994

Second AI winter

Limitations of if-then reasoning become more apparent.

1987: Market for Lisp machines (specialty hardware for running AI applications) collapses.

1987: DARPA again cuts funding for AI research.

1990: Expert systems, an attempt to replicate human reasoning through a series of if-then rules, failed. The software proved hard to maintain and couldn't handle novel information, resulting in a cutback in AI development.

1991: Japanese Ministry of International Trade and Industry's Fifth Generation Computer project failed to deliver on goals of holding conversations, interpreting images and achieving humanlike reasoning.

Could a 3rd AI Winter occur (soon)?

Is AI a good or bad thing?

“AI will be either the best, or the worst thing, ever to happen to humanity.” – Stephen Hawking

Hawking: “I Fear That AI May Replace Humans Altogether”

He talked about the **Technological Singularity**: Machines smart enough to think better than humans.

In terms of when that will happen (if ever) Hawking didn't exactly offer a time table.

But, SoftBank CEO Masayoshi Son and Google chief engineer Ray Kurzweil, have put the timeframe for the singularity even sooner — within the next 30 years.

Ray Kurzweil textbook: Singularity is Near (2029!)

Can machines think? – Turing Test / Gen AI (LLMs)

In 1950, Alan Turing posed the question “**Can machines think?**” ‘Thinking’ is **difficult to define**, he proposed a simpler question, such as whether a **machine could imitate a human** during a conversation with another human.

Known as the **Turing Test**, this involves a human judge asking questions to an unknown party in another room, who may be either human or machine, to evaluate whether they are human or not. The machine passes the test if it can convince the judge into believing it is human.

The Turing Test was significant because it provided the first solid benchmark to answer the question “**Can machines do what we humans (as thinking entities) do?**”

AI is Born – Hot Summer of 1995

The field of AI research was officially born in the summer of 1955 at the Dartmouth Conference which introduced the term “artificial intelligence” to unify the various research efforts in cybernetics, automata theory, and complex information processing to give machines the ability to “think”.

A small group of prominent researchers including John McCarthy, Marvin Minsky, Claude Shannon and Norbert Wiener proposed that “*every aspect of learning or any other feature of intelligence can be so precisely described that a machine can be made to simulate it.*” This provided a clear pragmatic direction for subsequent AI research.

Perceptron – The 1st Artificial Neural Network

In 1958, Frank Rosenblatt created the **perceptron** learning algorithm, the **simplest type of neural network with only one layer of neurons connecting inputs to outputs.**

The New York Times reported the perceptron to be “*the embryo of an electronic computer that the Navy expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.*” However, it was proven that the single layer perceptron could only recognize **linearly separable patterns**, but not more complex types such as the XOR (exclusive OR) function.

The field of neural networks stagnated for a decade, until it was realized that **multi-layer perceptrons**, with sufficient **computing power** and **data**, were a very effective way of modelling more complex **non-linear functions**.

This is the **principle behind current research in deep learning** with many complex layers of neurons

The Exclusive-OR (X-OR) problem

Minsky, Papert analyzed of Perceptron, concluded perceptrons only separated linearly separable classes. Papert gave birth (X-OR) problem

Classical problem in AI, one of the reasons for winter of AI during 70s.

Perceptron is based on the simplification on neuron architecture as proposed by McCulloch–Pitts, termed as McCulloch–Pitts neuron

McCulloch-Pitts Neuron



a	b	c
0	0	0
0	1	1
1	0	1
1	1	1

OR Function, Threshold=1

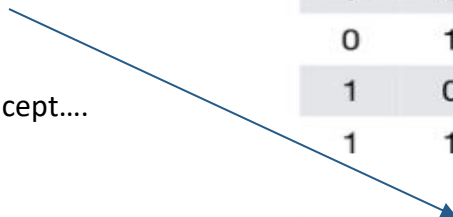


a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

XOR Function, Threshold=?

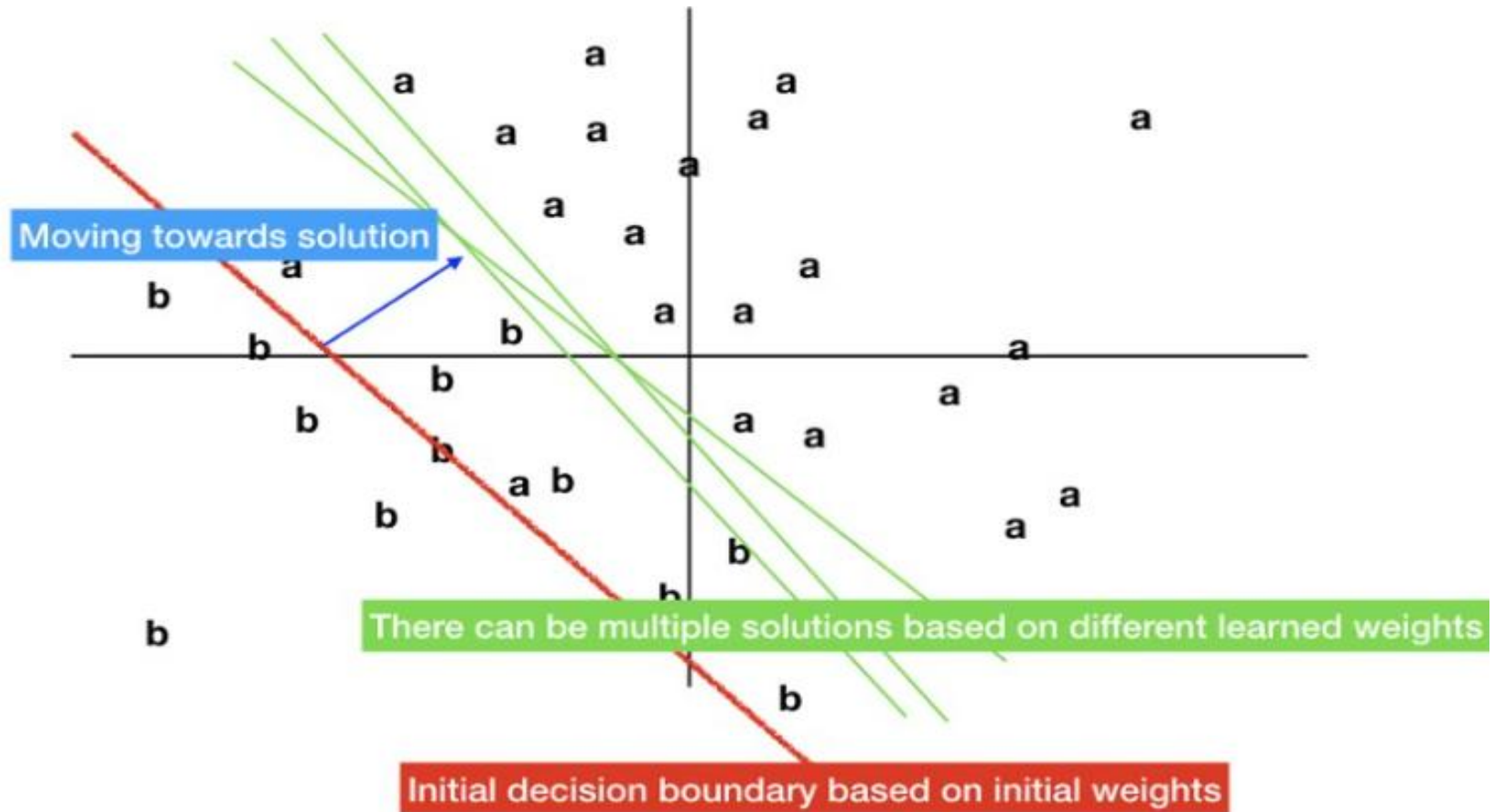
Threshold Function

A mathematical concept....



Perceptron Learning

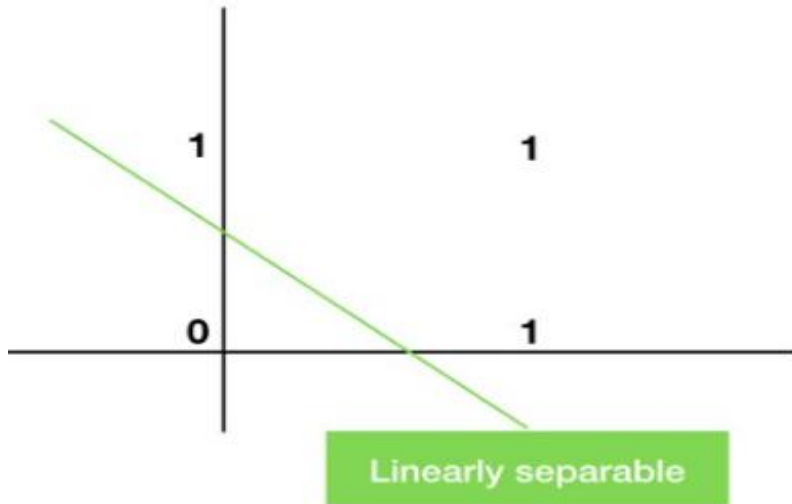
Minsky & Papert used the simplification of Perceptron to prove that it is **incapable of learning very simple functions**. Learning by perceptron in a 2-D space is shown below:



The XOR Problem – The curse for Data Scientists

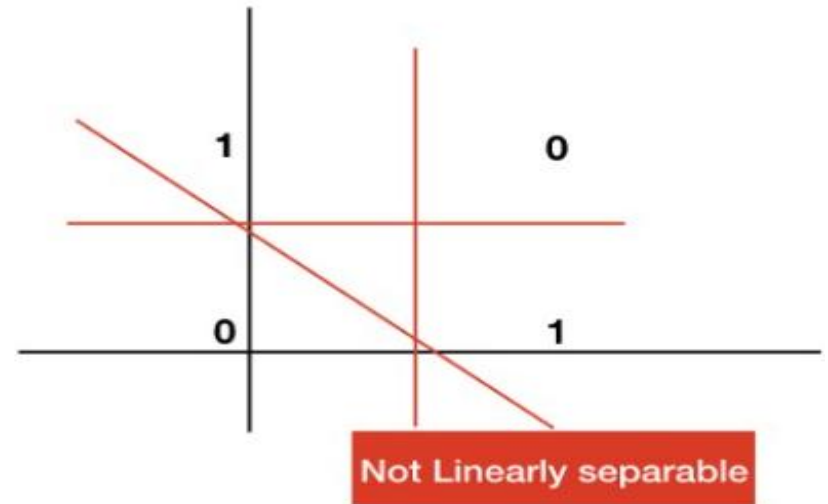
They chose Exclusive-OR as one of the example and proved that Perceptron doesn't have ability to learn X-OR. As described in image 3, X-OR is not separable in 2-D:

Inclusive-OR



a	b	c
0	0	0
0	1	1
1	0	1
1	1	1

Exclusive-OR



a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

Deep Learning: Extension of basic Perceptron model

Deep Learning is one **extension of basic Perceptron** model, in which we create **stack of neurons** and arrange them in **multiple layers**.

Initial models with single hidden layers were termed **multi layer perceptrons (MLPs)** and are considered **shallow networks**.

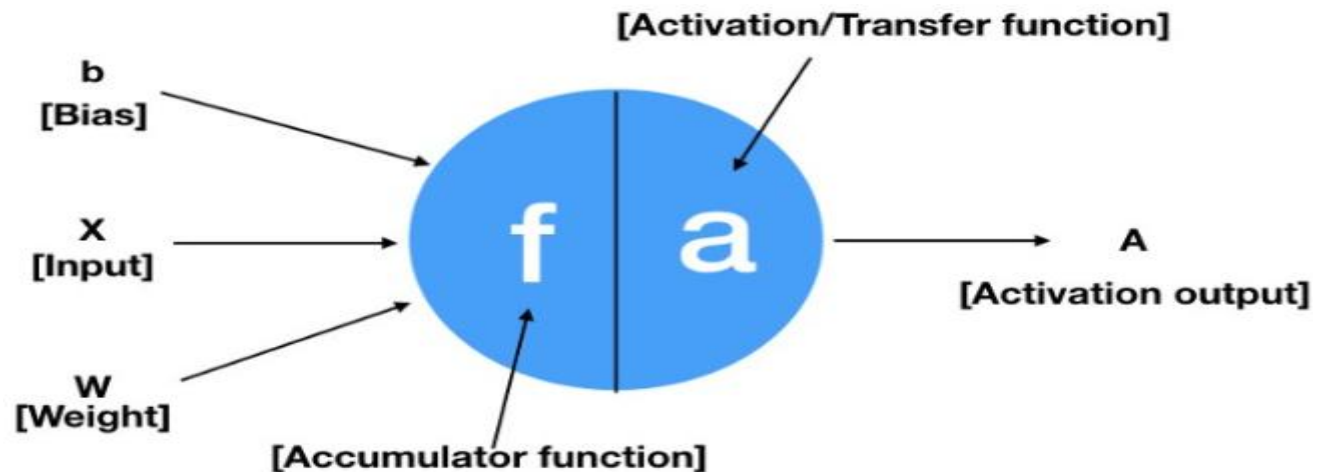
Deep networks have multiple layers and in recent works have shown capability to efficiently solve problems like object identification, speech recognition, language translation and many more.

While neural networks were inspired by human mind, the **Goal in Deep Learning** is not to copy human mind, but to **use mathematical tools to create models which perform well in solving problems** like image recognition, speech/dialogue, language translation, art generation

Single Neuron

Each **neuron** fed with an **input** along w/ associated **weight** and **bias**. A neuron has two functions:

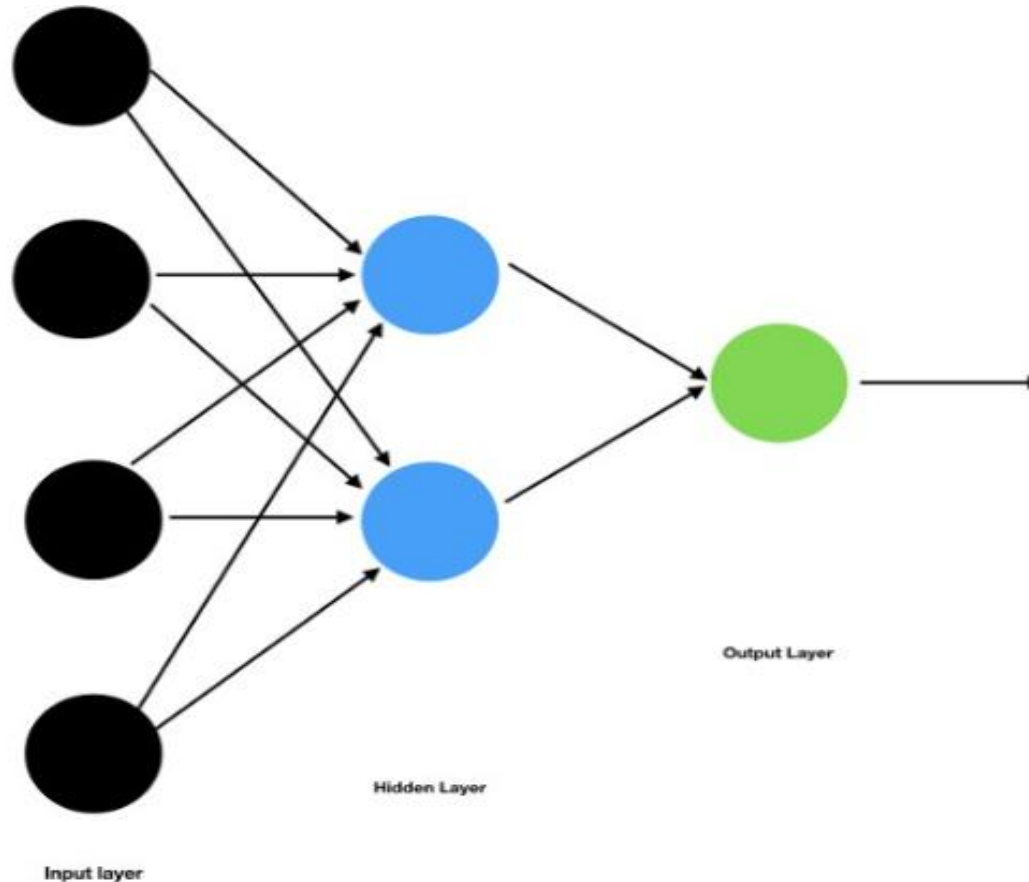
- 1) **Accumulator function**: It essentially is the **weighted** sum of input along with a **bias** added to it.
- 2) **Activation function**: Activation functions are non-linear function. And as the name suggests is a function to decide whether **output of a node will be actively participating in the overall output** of the model or not. **ReLU** (Rectified Linear Unit) is the most popular activation function.



Equations:
 $f = WX + b$
 $a = \text{sigmoid}(f)$
 $A = a$

Multi-Layer Perceptron (MLP)

Describe a process of **solving X-OR** with the help of **MLP** with one hidden layer. So, our model will have an **input layer**, one **hidden layer** and an **output layer**. Model will look something this:



Multi Layer Perceptron

MLP Model – Input Layer

Deep Learning models use mathematical tools to process input data.

Input layer must be represented w/ data in form of numbers.

In the input data we need to focus on two major aspects:

- ❓ **Number of features (columns):** Input given to a learning model may have only single feature which impacts the output. But, in most cases output depends on multiple features of input (e.g. object identification in a color image considers RGB values associated with each pixel.)
- ❓ **Number of examples (rows):** For each problem we have to feed network multiple input examples so that it can **generalize** over problem space. (e.g. if we wish to develop a model which identifies cats, we would require thousands of dog images in different environments, postures, images of different dog breeds)

Having m #of examples, n # of features, a $m \times n$ matrix is formed.

X-OR Inputs & Outputs

- ❑ The input is arranged as a matrix where rows represent examples and column represent features. So, if we have say **m** examples and **n** features then we will have an **m x n** matrix as input.
- ❑ In X-OR, have 4 examples and 2 features, so input is a **4 x 2** matrix

Examples[#4]	Inputs		Output
	a	b	c
	0	0	0
	0	1	1
	1	0	1
	1	1	0
Features[#2]			

XOR Problem – Input Data & Output Data

In Keras, define input & expected output with following lines of code:

`x = np.array([[0,0],[0,1],[1,0],[1,1]])`

`y = np.array([0,1,1,0])`

```
>>> import numpy as np
>>> x = np.array([[0,0],[0,1],[1,0],[1,1]])
>>> y = np.array([0,1,1,0])
>>> print(x)
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
>>> print(y)
[0 1 1 0]
```

MLP Model – Output Layer

Based on problem at hand, expect different kinds of output:

- ❑ **Two-class (binary)** classification
- ❑ **Multi-class** classification
- ❑ Advanced tasks like language translation, text summary generation have complex (**probabilistic**) output space

The activation function in **output layer** is selected based on the output space. In binary classification, **sigmoid** activation is best choice, multi class classification **softmax** is most popular choice

X-OR Input & Output Layers – Keras Example

In X-OR problem, output is either 0 or 1 for each input sample. It is a two-class or binary classification problem. Use **binary cross entropy** (loss function) along with **sigmoid activation function** at output layer.

In Keras we define output layer as follows:

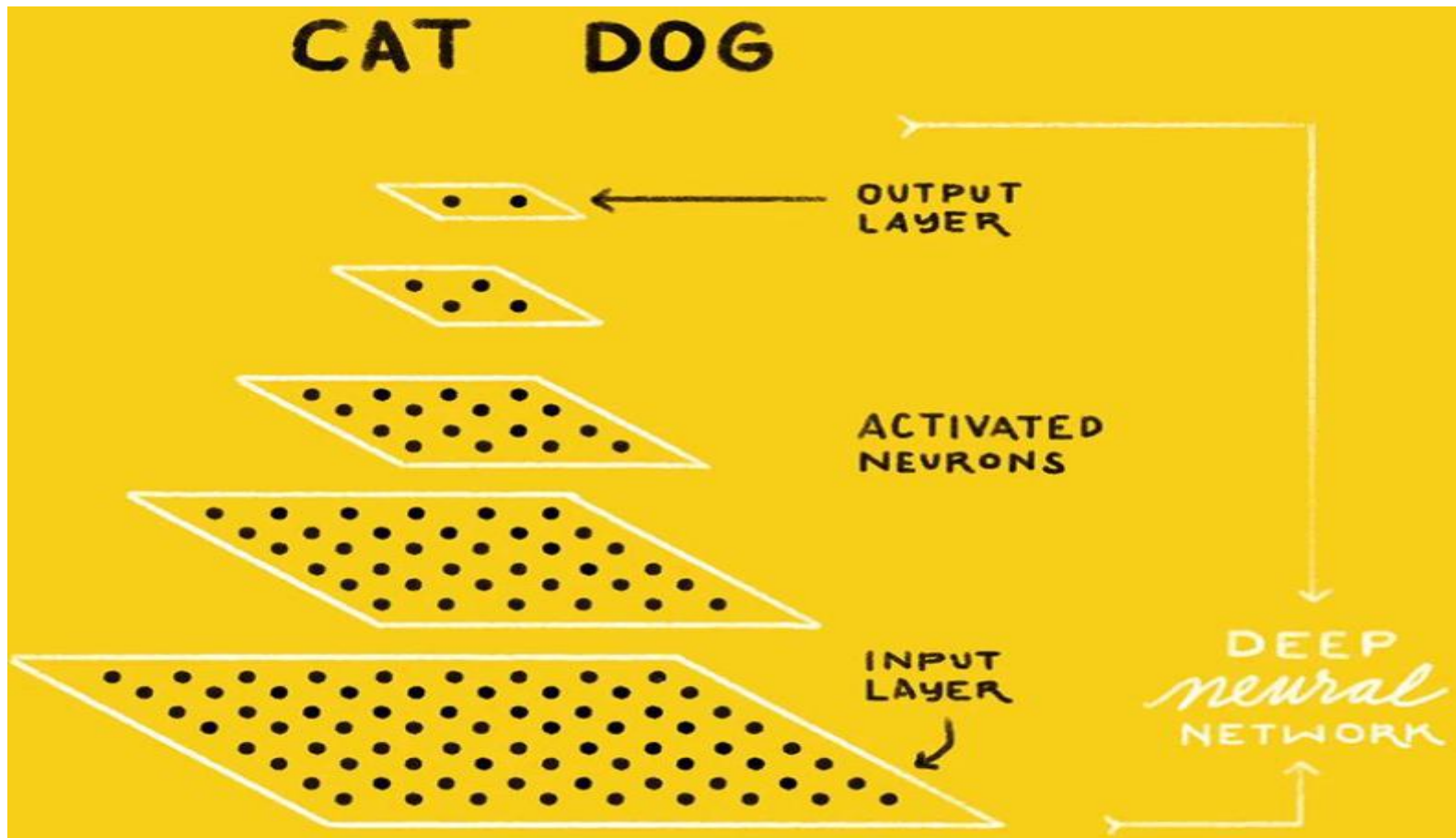
```
model.add(tf.keras.layers.Dense(units=2,activation=tf.nn.relu,input_dim=2))  
model.add(tf.keras.layers.Dense(units=1,activation=tf.nn.sigmoid))
```

```
model.compile(loss='binary_crossentropy',  
              optimizer='adam',  
              metrics=['accuracy'])
```

Deep Learning – Hidden Layers

A DL network can have **multiple hidden (deep) units**.

Purpose of hidden units: learn hidden feature or representation of input data which eventually helps in solving the problem at hand.



E.g., cat recognition: hidden layers, **1st** find the edges, **2nd** hidden layer may identify body parts, **3rd** hidden layer may make prediction whether it is a cat or not

MLP Model – Hidden Layer

MLP has one hidden layer. It is a shallow network.

Expect: Hidden layer to transform input of X-OR from 2-D plane to another form where can find a separating plane matching our expectation for X-OR output. Use ReLu activation function in hidden layer to transform input data. Transformation is shown:

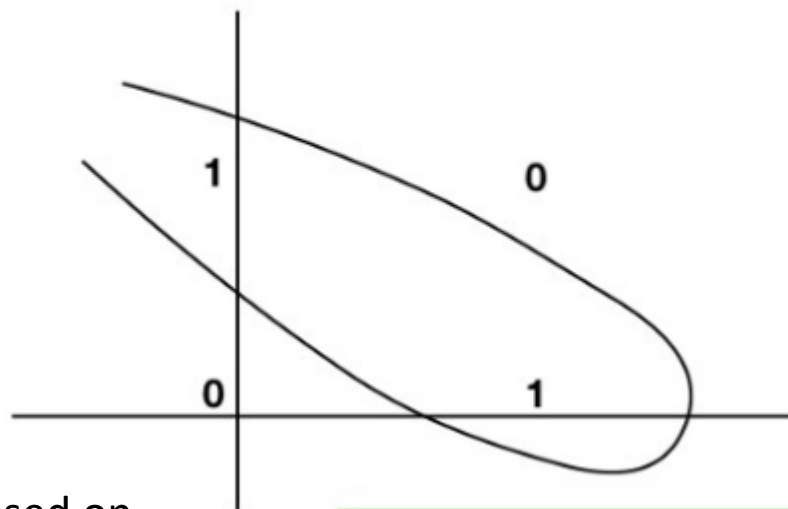
See slide #40
for Keras code....

Hidden layer has 2 units
Uses ReLu as activation.
Input to hidden unit is 4 examples
each having 2 features. ie a 4x2
matrix.

Hence dimensions of associated
weight matrix (W) would be 2x2

$$\mathbf{O} = \mathbf{W} * \mathbf{X} + \mathbf{b}$$

Solving X-OR



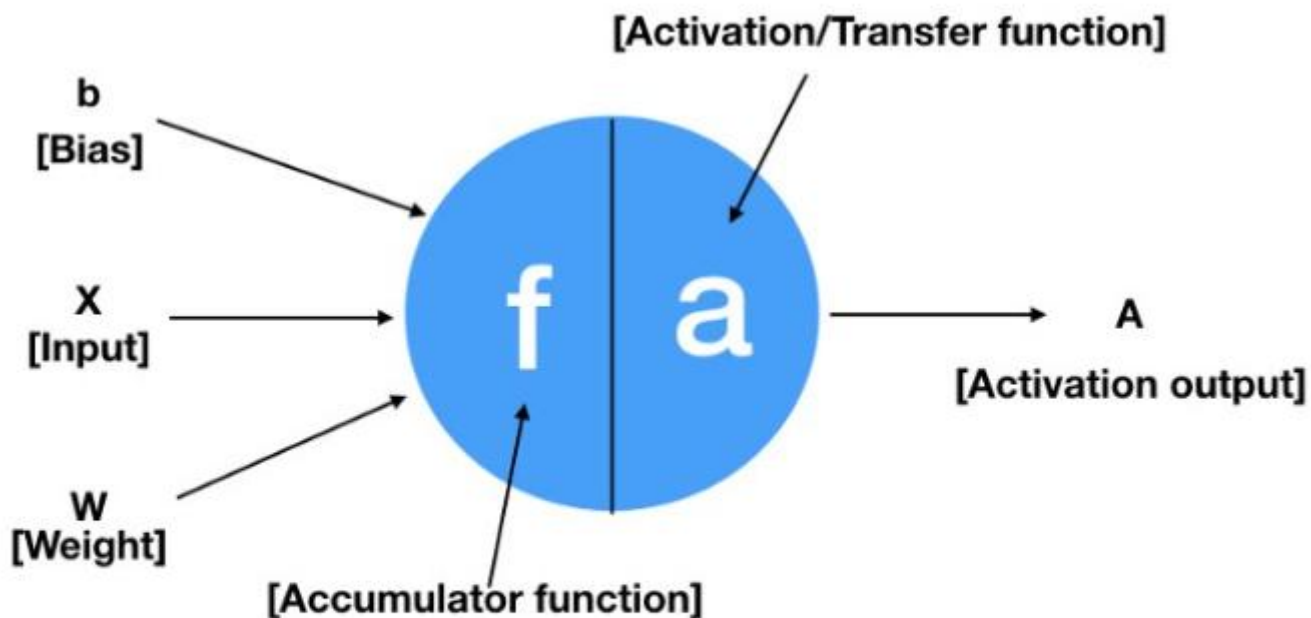
Non Linear separating plane

Shape of W determined based on
the **input** and **output shapes**.

Neural Networks – Weights and Biases

All input, hidden layers in NNs have associated **weights** and **biases**. Weights and biases are the values which move the solution boundary in solutions space to correctly classify the inputs.

Weights are randomly initialized and biases are all set to zero.



Equations:

$$f = WX + b$$

$$a = \text{sigmoid}(f)$$

$$A = a$$

Weights & Biases – XOR Problem Values

For X-OR values of **initial weights and biases** are as follows (set randomly by Keras implementation during my trial, your system may assign different random values).

Obtain weight value in keras using ***model.get_weights()*** function.

- ❑ ***Hidden Layer weights:*** `array([[ 0.6537529 , -1.0085169 ], [ 0.11241519, 0.36006725]], dtype=float32)`
- ❑ ***Hidden Layer bias:*** `array([0., 0.], dtype=float32)`
- ❑ ***Output Layer weights:*** `array([[ -0.38399053], [ -0.0387609 ]], dtype=float32)`
- ❑ ***Output layer bias:*** `array([0.], dtype=float32)]`

Deep Learning – Loss & Cost Functions

For model to generalize over input space and to make it capable of predicting accurately for new unseen cases, we require to train the model with available inputs.

During training, predict the output of model for different inputs and compare the predicted output with actual output in our training set

The **difference in actual and predicted output** is termed as **loss**.

The **summation of losses across all inputs** is termed as **cost function**

Selection of a loss and cost functions depends on the kind of output we are targeting.

Loss & Cost Functions – XOR Problem

For **classification** we use **cross entropy cost function**.

In Keras we have:

- *binary cross entropy* cost function for binary classification, and
- *categorical cross entropy* function for multi class classification.

Compile XOR Problem model in Keras as follows:

```
model.compile(loss='binary_crossentropy',  
              optimizer='adam',  
              metrics=['accuracy'])
```

Deep Learning – Back Propagation

The **goal of training** is to **minimize the cost function**.

This is achieved using **back propagation algorithm**.

Back propagation algorithm is a milestone in neural networks!

Back Propagation allows the gradients to back propagate through the network and then these are used to **adjust weights and biases** to move the solution space towards the direction of **reducing cost function**.

We'll revisit and deep dive into Back Propagation...

Training / Fit the Model

Training in Keras is started with following line:

```
model.fit(x,y,epochs=1000,batch_size=4)
```

Running 1000 iterations (**epochs**) to fit the model to given data.

Batch size is 4 i.e. full data set as our data set is very small. In practice, we use very large data sets and then defining batch size becomes important to apply **stochastic gradient descent** (sgd)

Epoch – Batch Size – Iterations

In the neural network terminology:

One **epoch** = one forward pass and one backward pass of all the training examples

Batch Size = the number of training examples in one forward/backward pass. The higher the batch size, the more memory space you'll need.

Number of Iterations = number of passes, each pass using [batch size] number of examples. To be clear, one pass = one forward pass + one backward pass (we do not count the forward pass and backward pass as two different passes).

Epoch – Batch Size – Iterations (Example)

Epochs

One Epoch is when an ENTIRE dataset is passed forward and backward through the neural network only ONCE.

Note-1: One epoch leads to underfitting of the curve in the graph

Note-2: One epoch is too big to feed (memory) at once we divide it in several smaller batches.

Batch Size

Total number of training examples present in a single batch.

Note: Batch size and number of batches are two different things.

Number of Iterations

Iterations is the number of batches needed to complete one epoch.

E.g. have 2000 training examples.

Divide the dataset of 2000 examples into batches of 500 then it will take 4 iterations to complete 1 epoch.

Where Batch Size is 500 and Iterations is 4, for 1 complete epoch.

XOR Problem Solution – JNB

Launch the JNB with complete TF/Keras code to solve XOR problem...

Questions?

